Project
Performance oriented programming (sequential)

J. Daniel García

1 Objective

This project's fundamental goal is to help students understand performance in sequential programs and become familiar with performance optimization. In addition, they will become familiar with performance evaluation techniques.

Specifically, the project will focus on developing sequential software in C++ using features up to C++23.

2 Project description

To carry out this project, you must implement in C++ a gravitational simulation application for a set of objects in a two-dimensional space.

Important

The program must only use sequential programming techniques without making use of any kind of concurrency or parallelism.

Two versions of the program will be implemented using the arrays of structures (**aos**) and structures of arrays (**soa**) techniques.

For a successful execution of the project, the following sections of this document provide the requirements that must be satisfied by the developed programs.

2.1 The gravitational interaction problem

The problem you must solve consists of the simulation over time of the positions of a set of objects in an enclosure. This computation is performed during the program execution at regular time intervals with length Δt.

The program must perform the simulation in a 2D space. We consider the space a closed enclosure. Consequently, when an object impacts a side in the border of the enclosure, a rebound happens, changing the direction of the object motion.

Finally, in every time increment Δt possible collisions between different objects are checked.

In this way, it is expected that students simulate the behavior of N different objects moving in a 2D space.

CC BY-NC-SA 4.0

2.1.1 Particle properties

Each object in the simulation has the following properties:

- **Mass** (m): The mass of the object.
- **Radius** (r): The radius of the object, used for collision detection.
- **Position** ($\vec{p}$): The position vector of the object in the 2D space, with coordinates x and y.
- **Velocity** ($\vec{v}$): The velocity vector of the object in the 2D space, with components v_x and v_y.
- **Acceleration** ($\vec{a}$): The acceleration vector of the object in the 2D space, with components a_x and a_y.

2.1.2 Attraction forces computation

Recommendation

Students are recommended to follow the described order in operations to make sure they get correct results.

- **Gravitational force**: To compute the gravitational force F_{ij} between two objects, equation 1 is used. Gravitational constant (G), object masses of both objects (m_i and m_j), and position vectors ($\vec{p_i}$ and $\vec{p_j}$) will be used.

$$\vec{F_{ij}} = \frac{G \cdot m_i \cdot m_j}{||\vec{p_j} - \vec{p_i}||^2} \cdot \frac{(\vec{p_j} - \vec{p_i})}{||\vec{p_j} - \vec{p_i}||} = \frac{G \cdot m_i \cdot m_j \cdot (\vec{p_j} - \vec{p_i})}{||\vec{p_j} - \vec{p_i}||^3} \tag{1}$$

Where the norm of the difference vector between two position vectors $\vec{p_i}$ (with coordinates x_i and y_i) and $\vec{p_j}$ (with coordinates x_j and y_j) is given by equation 2.

$$||\vec{p_j} - \vec{p_i}|| = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \tag{2}$$

The universal gravitational constant will be the value $6.674 \cdot 10^{-11}$.

Note

For a pair of objects i and j, the force applied to object i by object j is equal in magnitude and opposite in direction to the force applied to object j by object i.

With the convention in equation 1, the force on object **i** is $\vec{F}_{ij}$ and the force on object **j** is $\vec{F}_{ji} = -\vec{F}_{ij}$. You can avoid computing the force twice by only calculating it for one pair and then applying the opposite to the other.

An object does not apply any force on itself.

$$\vec{F}_{ji} = -\vec{F}_{ij}$$

$$\vec{F}_{ii} = 0$$

- **Acceleration vector**: Acceleration ($\vec{a}$) is determined by equation 3 using its mass (m) and all the force contributions.

$$\vec{a}_i = \frac{1}{m_i} \sum_{j=1}^{N} \vec{F}_{ij} \tag{3}$$

- **Velocity vector**: Once acceleration has been computed, new velocity ($\vec{v}^*$) is updated by equation 4 using its previous value ($\vec{v}$), acceleration ($\vec{a}$), and the time step (Δt).

$$\vec{v}_i^* = \vec{v}_i + \vec{a}_i \cdot \Delta t \tag{4}$$

- **Position vector**: New position ($\vec{p}^*$) is updated by equation 5 using its previous value ($\vec{p}$), the velocity ($\vec{v}$) and the time step (Δt):

$$\vec{p}_i^* = \vec{p}_i + \vec{v}_i \cdot \Delta t \tag{5}$$

- **Rebound effect**: If the position passes one of the enclosure bounds:

  1. The object is placed in the corresponding bound plane:

$$x - r \leq 0 \Rightarrow x \leftarrow r$$

$$y - r \leq 0 \Rightarrow y \leftarrow r$$

$$x + r \geq width \Rightarrow x \leftarrow width - r$$

$$y + r \geq height \Rightarrow y \leftarrow height - r$$

2. Besides the corresponding velocity component is normalized to point toward the enclosure interior:

$$x - r \leq 0 \Rightarrow v_x \leftarrow |v_x|$$
$$y - r \leq 0 \Rightarrow v_y \leftarrow |v_y|$$
$$x + r \geq width \Rightarrow v_x \leftarrow -|v_x|$$
$$y + r \geq height \Rightarrow v_y \leftarrow -|v_y|$$

2.1.3 Object collisions

When two or more objects collide they collapse into a new object. Two objects are considered to collide when their distance is less than the sum of their radii. For checking collision purposes, the distance between the objects is computed using their position vectors.

The distance between two objects i and j is computed as:

$$d_{ij} = \|\vec{p}_i - \vec{p}_j\| \tag{6}$$

After a collision, designate the absorbing object as object i and the absorbed object as object j. The object with greater mass absorbs the other; if their masses are equal, the object with greater radius absorbs the other; if both mass and radius are equal, the object with the smaller identifier absorbs the other.

In this case both objects (i and j) lead to a single object (k) with the following properties:

- New mass results from mass addition ($m_k = m_i + m_j$).
- New radius is the radius of the absorbing object.
- New position is the position of the absorbing object.
- New velocity results from velocity addition ($\vec{v}_k = \vec{v}_i + \vec{v}_j$).
- Absorbing object is replaced by object k.
- Absorbed object no longer exists.

Important

Keep in mind that in those cases there is one fewer object in the system.

Collision checks must be performed at the end of each simulation iteration.

Additionally, this check must also be performed before starting the simulation.

Each object gets initially an integer consecutive id (starting at 0). When a collision occurs and a new object is created, it receives the id of the absorbing object.

Collision checks must examine object pairs in lexicographic order of their identifiers. Whenever a collision is detected, resolve it and restart the scan from the lowest-identifier remaining object to ensure that no further collisions are missed.

2.2 Requirements

2.2.1 Program execution and input parameters

- Teams will develop two programs named **grav-soa** (structure of arrays version) and **grav-aos** (array of structures version).
- The program will take the following parameters for its correct execution:
  - **num_iterations**: integer number, greater than 0, indicating the number of iterations (time steps) that will be simulated.
  - **width**: real positive number indicating the width of the simulation enclosure.
  - **height**: real positive number indicating the height of the simulation enclosure.
  - **time_step**: real positive number indicating the time increment for each iteration of the simulation.
- When the number of parameters is wrong, the program will terminate with an error code (see **EXIT_FAILURE**) and an error message to the standard error output.

```
user@machine:~$
user@machine:~$./grav-soa 100 1000 500
grav-soa: grav-soa: invoked with 3 arguments
        Expected 4 arguments: <num_iterations> <width> <height> <time_step>
        Received arguments: ["100", "1000", "500"]
```

- When any parameter is wrong, the program will terminate with an error code (see **EXIT_FAILURE**) and an error message to the standard error output.

```
user@machine:~$
user@machine:~$./grav-soa 100 -1000 2000 0.25
grav-aos: Invalid value for width: -1000
```

- The program will read from file **input/input.txt** the initial configuration of the simulation, including the masses, radii, positions, and velocities of all objects. This will be a sequence of lines, one per object with the following values:
  - mass (floating point double precision).
  - radius (floating point double precision).
  - position (x, y coordinates in floating point double precision).
  - velocity (x, y components in floating point double precision).

  The input will be read until the end of the file (EOF) is reached. If a line does not conform to the expected format, it will be ignored. If the last line does not conform to the expected format or it is incomplete, it will be ignored.
- If any object has an invalid initial configuration, that object will be discarded. The following conditions will be checked:
  - Mass must be positive.
  - Radius must be positive.
  - Coordinate X must be between the radius and the width of the enclosure minus the radius.
  - Coordinate Y must be between the radius and the height of the enclosure minus the radius.

- Each valid object gets a consecutive integer identifier (starting at 0).
- Once all input parameters have been processed and initial positions and velocities have been processed for all simulation elements, collision processing is performed (see section 2.1.3) and the program will write this information to a file named **init_config.txt**, with the following format:
  - All real values will be printed with three decimal places in the fractional part.
  - A header line including:
    * Width of the enclosure.
    * Height of the enclosure.
    * Time step.
    * Number of initially valid objects after initial collision processing.
  - One line per object where mass, radius, position (x, y coordinates), velocity (x, y components) must be written. These six values must be separated by blanks.
  - Example for an enclosure of size 1000.0 × 2000.0, with time step 0.250 and 2000 initially valid objects.

```
1000.000 2000.000 0.250 2000
125.720 2.540 10.500 12.250 43.225 75.325
200.300 15.000 50.000 60.000 5.000 10.000
...
```

Note

This is just an example of file format. Content data does not need to correspond to those obtained with the example program and input parameters given in the example.

2.2.2 Data storage

Once all simulation iterations have finished, the program shall store final data in a file named **final_config.txt**, using the same format as **init_config.txt** and reporting the number of objects remaining after collisions.

3 Task

3.1 Software Development

3.1.1 Program to be developed

This task consists of developing a sequential version of the described application using C++23.

Important

Please note that this version does not allow the use of multiple execution threads (*multithreading*) or any kind of parallelism.

Two independent executable programs will be developed: **grav-soa** and **grav-aos**, which will respectively implement the **SOA** and **AOS** strategies.

- **AOS – Arrays of Structures**: The particles will be represented as a single sequence of objects, where each object contains all the properties describing a particle.
- **SOA – Structure of Arrays**: The particles will be represented as multiple independent sequences. Each of the sequences will contain elements corresponding to a specific property of the particles.

Recommendation

- Evaluate the different sequence containers offered by the standard library. You can find a list of them at https://en.cppreference.com/w/cpp/container.
- Consider the memory layout and access patterns of your data structures, as they can significantly impact performance.

3.1.2 Project structure

A project template will be provided for you to start your development.

This repository contains the following characteristics:

- The **cmake** directory contains a file with additional utilities that will be useful for performing coverage analysis of your unit tests.
- The **common**, **aos** and **soa** directories are designed for you to include the source code you must develop.
- The **utcommon**, **utaos** and **utsoa** directories are designed for you to include the source code of your unit tests.
- In the root directory you will find, among others, the following files:
  - Configuration files **.clang-format** and **.clang-tidy**. Please do not modify these files unless you receive explicit instructions to do so.
  - File **CMakeLists.txt** with the project build instructions.
  - File **CMakePresets.json** with the predefined configuration of compilation options.

Project configuration: main CMake file The main **CMakeLists.txt** file establishes the following options:

- Minimum CMake version: 4.0.
- Languages to use in the project: exclusively C++.
- No external libraries are allowed (unless approved by your instructor), with the exception of the following libraries:
  - **GSL** (C++ Core Guideline Support Library). This library offers utilities that extend the standard library and that may be useful to you.

  - **GoogleTest**. You can use this library to prepare your unit tests. Alternatively, you may use another testing framework allowed by the instructor.

  During the compilation process, these libraries are downloaded and configured.

- An **ENABLE_CLANG_TIDY** option is established that can be enabled and disabled. When this option is active, the rules established by the **clang-tidy** tool are applied to all source code in the project.
- The **common**, **soa** and **aos** directories are added, as well as the corresponding unit test directories **utcommon**, **utaos** and **utsoa**.

Important

All source files must compile without problems and will not emit any compiler warnings.

They must also compile without any problems when the **ENABLE_CLANG_TIDY** option is enabled.

Project Configuration: *presets* files The **CMakePresets.json** file, which is also supplied for download, contains several sets of pre-established options:

- Configuration options:
  - **default**: This is the default configuration used for compiling without running **clang-tidy**.
  - **clang-tidy**: Enables **ENABLE_CLANG_TIDY** on top of the default configuration.
- Build options:
  - **gcc-debug**: Debug mode compilation with default options.
  - **gcc-release**: Optimized mode compilation with default options.
  - **clang-tidy-debug**: Debug mode compilation with default options. It also performs static code analysis.
  - **clang-tidy-release**: Optimized mode compilation with default options. It also performs static code analysis.

The options that all configuration and compilation profiles established in this file share are the following:

- Use of the internal **Ninja Multi-Config** build generator.
- Compilation options with C++ modules are disabled.
- Use of the **g++-15** compiler.
- Compile in strict C++23 mode.
- Warning and error options for a stricter compilation are enabled.

Project Compilation Compilation can be carried out in **Debug** mode (presets **gcc-debug** and **clang-tidy-debug**) or **Release** mode (presets **gcc-release** and **clang-tidy-release**).

Important

Remember that all evaluations will be carried out with compiler optimizations enabled, using the **gcc-release** compilation profile. If you deem it appropriate, you can add additional optimization options in the **CMakePresets.json** file, and stating it in the project report.

Code Quality Rules The source code must be well-structured and organized, as well as appropriately documented. It is recommended (although not required) to follow the **C++ Core Guidelines** (http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines).

Nevertheless, all rules specified in the document **Coding rules for the C++ language** which is published separately must be followed.

To facilitate the work of the teams, a configuration file for the **clang−tidy** tool will also be supplied.

3.1.3 Software components

The development performed is structured into three software components, which are placed in the existing folders **common**, **aos** and **soa**.

Common folder This folder contains all source files with data types and functions that are common to both versions of the program. In turn, it contains two subdirectories **include** (with all header files) and **src** (with all **.cpp** files).

The **CMakeLists.txt** file defines a library called **common**. In the code that is supplied, the **target_sources** directive depends only on the file **src/vector.cpp**. You must add to this list all source files that you want to include in the **common** library.

The **common** target uses the **include** directory as the include directory. Additionally, a dependency on the **GSL** library is established.

AOS folder This folder contains all source files with data types and functions that are specific to the **AOS** version. It contains two subdirectories **include** (with all header files) and **src** (with all **.cpp** files).

Additionally, in the **src** directory there must be a **main.cpp** file that will contain exclusively the **main()** function.

The **CMakeLists.txt** file defines an executable called **grav−aos**. In the code that is supplied, the **target_sources** directive depends only on the file **src/main.cpp**. You must add to this list all source files that you want to include in the **grav−aos** executable.

The **grav−aos** target uses **include** as the include directory. Additionally, dependencies on the **GSL** and **common** libraries are established.

SOA folder This folder contains all source files with data types and functions that are specific to the **SOA** version. It contains two subdirectories **include** (with all header files) and **src** (with all **.cpp** files).

Additionally, in the **src** directory there must be a **main.cpp** file that will contain exclusively the **main()** function.

The **CMakeLists.txt** file defines an executable called **grav—soa**. In the code that is supplied, the **target_sources** directive depends only on the file **src/main.cpp**. You must add to this list all source files that you want to include in the **grav—soa** executable.

The **grav—soa** target uses **include** as the include directory. Additionally, dependencies on the **GSL** and **common** libraries are established.

3.1.4 Unit Tests

A set of unit tests will be defined and also submitted. The use of GoogleTest (https://github.com/google/googletest) is recommended. If another framework for unit testing is desired, authorization from the course coordinator must be obtained.

In any case, evidence of sufficient unit tests must be provided.

utcommon tests The **utcommon** directory contains all unit tests that correspond to the components of the **common** library.

In **CMakeLists.txt** two variables are defined:

- **COMMON_SRC_FILES**: Contains the list of source files being tested.
- **CURRENT_DIR_SRC_FILES**: Contains the list of source files that contain the tests that should be executed.

As an example, a test file **test_vector.cpp** is presented with tests for the **vector** type.

utaos and utsoa tests They contain the tests for the corresponding **aos** and **soa** components, following a similar structure.

3.1.5 Use of AI Tools

The use of AI-based tools during the project is permitted. However, the following must be taken into account:

- If you use an AI-based tool, you must declare its uses in the design section of the project report. No penalty will be incurred for the declaration as long as a statement of use is included.
- No support will be given for the use of such tools. In particular, if you have questions about your code, you must be capable of explaining said code.
- Keep in mind that some AI tools can generate insecure code or code with low performance.
- You may be required to provide an explanation of your own code.

3.2 Performance and Energy Evaluation

This task consists of carrying out a comparative evaluation of performance and energy consumption. To carry out the performance evaluation, the total execution time will be measured using the **perf** tool. In addition, energy will also be measured and power will be derived.

All performance evaluations will be carried out on a node of the **avignon** cluster.

Plot the total execution times, energy usage, and power for different scenarios. Perform a scalability analysis for those application options for which it makes sense.

The project report will include conclusions derived from the results. Please do not limit yourself to a mere description of the data. Try to find convincing explanations for these results.

4 Report structure to be submitted

The report should be limited to describing the work performed. Conciseness in descriptions will be especially valued. It must be submitted in PDF format. It must contain, at least, the following sections:

- **Title page**: will contain the following data:
  - Project name.
  - Assigned team number.
  - Name and NIA of all authors.
- **Design**. It should explain the global design of the application, complementing it with design diagrams using some commonly used notation (e.g. structure diagrams, behavior diagrams, ...).

  It must also include the main design decisions.

  Important

  Do not repeat in this section what can be specified in code comments, such as what are the parameters of a function, or its algorithm.

  Maximum length: 3 pages.
- **Optimization**. It must contain a discussion of the applied optimizations and their impact.

  In particular, optimizations performed on the original source code should be indicated, as well as optimizations enabled with additional compilation flags that are deemed appropriate.

  Maximum length: 1 page.
- **Tests performed**: Description of the test plan performed to ensure correct execution. It must include unit tests, as well as functional system tests.

  In the report, only a general overview of the testing approach should be given. The list of specific tests and how to execute them should be found in execution scripts (**utest.sh**/**utest.py** and **ftest.sh**/**ftest.py**).

  Maximum length: 2 pages.
- **Performance and energy evaluation**: It should include the performance and energy evaluations carried out.

  Maximum length: 5 pages.
- **Work organization**: It should describe the work organization among team members, making explicit the tasks carried out by each person.
  - It must contain a division of the project into tasks.
  - Tasks must be small enough so that a single person can be assigned to a task.
  - All team members must make relevant contributions in the software development process.
  - No more than one person can be assigned to a task. In such case, the task must be subdivided into subtasks.

  - The time dedicated by each person to each task must be indicated.

  Maximum length: 2 pages.

  A table in ODS format can be submitted with at least the following columns: **task name**, **person name**, **number of hours dedicated**. This file can be included as part of the **grav.zip** archive.

- **Conclusions**. Those derived from the performance evaluation results will be especially appreciated, as well as those that relate the work performed with the content of the course.

  Maximum length: 2 pages.

5 Submission procedure

The project report submission will be delivered through a submitter enabled for this purpose in Aula Global. In this submitter a single file that must be called **report.pdf** will be placed.

The entire project developed in a private GitHub repository will also be delivered in a ZIP file through Aula Global in a submitter enabled for this purpose. In this submitter a single compressed file in ZIP format that must be called **grav.zip** will be placed.

6 Grading

Final grades for this project are obtained in the following way:

- Performance: 20%.
- Energy use: 20%.
- Unit tests: 7%.
- Functional tests: 3%.
- Design quality: 5%.
- Code quality: 5%.
- Performance and energy evaluation in the report: 15%.
- Contributions from each team member: 20%.
- Conclusions: 5%.

Important

- If the submitted code does not compile, the final grade for the project will be **0**.
- If code contains unauthorized modifications to provided CMake files, the final grade for the project will be **0**.
- If numerous quality coding rules are ignored without justification, the final grade for the project will be **0**.
- If the execution time is deemed extremely high, the final grade for the project will be **0**.
- In case of copying all implied groups will get a grade of **0**. Moreover, the head of the school will be notified for the corresponding disciplinary actions.

The following are specific evaluation criteria for each of the evaluation dimensions. Please note that these criteria are **merely indicative** and are subject to the interpretation of your instructors.

Performance and energy Different evaluation scenarios and configurations with different levels of difficulty will be defined. For each evaluation, reference values will be established to obtain 5 points out of 10 in the evaluation. A threshold will also be established to obtain 0 points out of 10 in the evaluation.

The final grade for performance and energy will be obtained as the arithmetic mean of the grades from all tests.

Important: These reference values will be published at least one week before the submission deadline.

Unit tests The requirement is that there be a sufficient number of unit tests for all software developed using the GoogleTest framework (or some other testing framework allowed by your instructor). No specific coverage criteria have been defined, so coverage will not be analyzed. It will be considered that there are sufficient tests if there are tests for each function or member function of each class.

The following levels are defined:

- **Excellent**
  - There are three unit test executables generated: **utest-common**, **utest-soa** and **utest-aos**.
  - There is a script (**utest.sh**/**utest.py**) that launches all unit tests.
  - GoogleTest or another testing framework allowed by the instructor is used as the framework.
  - There are test cases for each function and for each member function that consider both basic cases and different error cases.
  - Unit tests do not depend on the existence of specific files in any directory.
- **Acceptable**
  - GoogleTest or another testing framework allowed by the instructor is used as the framework.
  - There is a set of tests that corresponds to each .cpp file of the source code, but there are no tests for all functions or member functions.

- **Insufficient**
  - GoogleTest or another testing framework allowed by the instructor is used as the framework.
  - There are some tests but they are insufficient.
  - Some segmentation fault or other catastrophic error occurs when executing the tests.
- **Deficient**
  - No unit tests using GoogleTest or another testing framework allowed by the instructor are provided.
  - There are no explanations of unit tests in the report.

Functional tests The requirement here is that there be evidence that functional tests of the complete application have been performed. An attempt should be made to automate the tests.

- **Excellent**
  - Complete functional tests exist and they consider both success cases and error cases.
  - Functional tests have been automated with some script and are reproducible.
- **Acceptable**
  - Functional tests exist but they are incomplete and only consider non-error cases.
  - Functional tests have been automated with some script and are reproducible.
- **Insufficient**
  - Functional tests are described in the report but no automated way to execute them is provided.
  - An attempt is made to perform functional tests from a program written in C++ instead of checking the executable functionality from a script.
- **Deficient**
  - There is no evidence of the existence of functional tests.
  - Functional tests are not mentioned in the report.

Design quality Design comprises logical design (data structures, classes, functions,...) and physical design (source code structure).

- **Excellent**
  - Diagrams are provided and they give a clear vision of the structure of the software.
  - The design of classes and functions is clearly justified.
  - The difference between the structures used for AOS and SOA is clear and these are adequate.
  - Auxiliary data structures are the most adequate.
  - The main decisions for each functionality and the performance-oriented design decisions, including optimizations, are justified.

  - Specific optimization flags used are mentioned.

- **Acceptable**
  - The design of classes and functions is justified, although there are unclear or unjustified decisions.
  - There is a difference between the structures used for SOA and AOS and these are adequate.
  - Auxiliary data structures are not very adequate.

- **Insufficient**
  - The design is not sufficiently justified and is difficult to understand.
  - The structures used for AOS and SOA are not adequate.

- **Deficient**
  - The application design is not described.

Code quality This section evaluates whether the code is of good quality and there are no bad practices. Most of these checks are already performed with **clang-tidy**.

- **Excellent**
  - The code is clear, concise and well formatted. The names of variables, functions and data types are consistent.
  - There are comments documenting each function and data type.

- **Acceptable**
  - The code is clear, concise and well formatted. The names of variables, functions and data types are consistent.
  - There are no comments documenting each function and data type.

- **Insufficient**
  - The code lacks clarity and readability.
  - There are no comments documenting each function and data type.
  - There are some violations of **clang-tidy**, but they are few.
  - Rules of **clang-tidy** have been disabled using **NOLINT** in an unjustified way, but they are few.

- **Deficient**
  - There are numerous violations of **clang-tidy** or rules of **clang-tidy** have repeatedly been disabled in an unjustified way (i.e., unauthorized **NOLINT**).

Performance and energy evaluation in the report This section will evaluate the discussion in the report about performance and energy evaluation.

- **Excellent**
  - A performance and energy evaluation has been carried out for the different program options.
  - A scalability analysis has been performed and graphs are provided.
  - Conclusions are included based on experimental data.
- **Acceptable**
  - A performance and energy evaluation has been performed for the different program options.
  - A scalability analysis has been performed for at least one significant parameter.
  - No graphs are provided, but data tables are provided.
  - Conclusions are included based on experimental data.
- **Insufficient**
  - A performance or energy evaluation has been performed only for some program options or only screenshots are provided without any graph.
  - No scalability analysis has been included.
  - There are no conclusions based on experimental data.
- **Deficient**
  - No performance and energy evaluation has been performed.

Individual member contributions This is the only section that is evaluated independently for each project team member.

Initial observation: All members of a team will receive a grade of **DEFICIENT** if any of the following circumstances occur:

- No division of work into individual tasks is presented.
- No list of completed tasks is presented.
- **Excellent**
  - Contributions are well described and have adequate granularity.
  - For each contribution, the person's dedication in hours is quantified. The contributions and their quantification are credible.
  - The person's contributions are adequate in quantity and quality.
- **Acceptable**
  - Contributions are well described and have adequate granularity.
  - For each contribution, the person's dedication in hours is quantified. The contributions and their quantification are credible.
  - The person's contributions are somewhat lower than expected in quantity and quality.

- **Insufficient**
  - Only the person's total dedication to the project in hours is quantified, but not the effort dedicated to each specific task.
  - The total number of hours dedicated by the person to the project cannot be determined.
- **Deficient**
  - The effort dedicated by the person to the project is not quantified in hours.

Conclusions This section evaluates the conclusions of the work.

- **Excellent**
  - Conclusions include what is inferred from the experimental data of the project and the statements are coherent and adequate.
  - Conclusions are presented in terms of performance and energy.
  - Conclusions about what was learned are included.
  - The work is connected with topics seen in the course.
  - The conclusions presented do not contain inaccuracies and are relevant.
- **Acceptable**
  - Conclusions are a mere repetition of experimental results without going further.
  - Conclusions about what was learned are included or the work is connected with topics seen in the course.
  - The conclusions presented do not contain inaccuracies and are relevant.
- **Insufficient**
  - Conclusions are a mere repetition of experimental results without going further.
  - No conclusions about what was learned are included nor is the work connected with topics seen in the course.
  - The conclusions presented contain slight inaccuracies, but in general are relevant.
- **Deficient**
  - There are no conclusions or they are irrelevant.